

Assistente de estudos com RAG

Apostila para entender, demonstrar e apresentar o projeto

Esta apostila explica o chat que consulta o material da disciplina Disruptive Architectures. Ela reúne a arquitetura implementada, o caminho dos dados, as tecnologias usadas, as decisões de segurança, as melhorias da interface, os erros encontrados e um roteiro de demonstração. O projeto está publicado: o site está no GitHub Pages e a API está no Vercel.

Em uma frase

O RAG procura trechos relevantes no material do curso, envia esses trechos ao modelo de linguagem e apresenta uma resposta baseada neles, com links para as fontes.

1. O que o projeto resolve

Um modelo de linguagem pode responder usando conhecimento geral e também pode inventar detalhes. Para um assistente de disciplina, a resposta precisa ficar ligada ao conteúdo realmente publicado pelo professor. O RAG, sigla de Retrieval-Augmented Generation, acrescenta uma etapa de busca antes da geração: a aplicação recupera evidências e as entrega ao modelo como contexto.

O objetivo deste projeto é permitir que um aluno faça perguntas em linguagem natural sobre aulas, laboratórios e conceitos, receba uma resposta em português e veja as páginas que sustentam essa resposta. Quando a busca não encontra evidência suficiente, o assistente deve reconhecer essa limitação em vez de inventar uma resposta.

2. Conceitos essenciais

Conceito	O que significa neste projeto
RAG	Busca informação externa antes de gerar a resposta. Une recuperação e geração.
Chunk	Trecho menor de uma página ou notebook. É a unidade que pode ser recuperada.
Embedding	Vetor numérico que representa características semânticas do texto.
Busca vetorial	Compara embeddings para encontrar trechos com significado próximo, mesmo sem repetir exatamente as palavras.
Busca lexical	Procura correspondência de termos no título, na seção e no texto armazenado no Oracle.
LLM	Modelo de linguagem que redige a resposta a partir da pergunta e das evidências recuperadas.
Fonte	Página do curso associada ao trecho usado e validada antes de virar link no chat.

Embeddings de documentos e de perguntas precisam usar o mesmo modelo. Caso contrário, os vetores ficam em espaços diferentes e a comparação deixa de ser confiável. Por isso a ingestão e a consulta usam o modelo multilíngue BGE-M3 da Cloudflare Workers AI, com vetores de 1024 dimensões.

3. Arquitetura implementada

O sistema tem duas partes publicadas separadamente. O GitHub Pages entrega o site estático do MkDocs e o widget do chat. O Vercel executa a API Python/FastAPI. O Oracle da FIAP mantém o texto e os dados das conversas. A Cloudflare fornece embeddings e busca vetorial. Gemini é o provedor principal de geração; Groq pode atuar como alternativa para limites ou indisponibilidade temporária do Gemini.

Etapa	Componente	Responsabilidade
1. Interface	MkDocs Material + chat-widget.js	Coleta a pergunta, mantém a sessão da aba e exibe resposta e fontes.
2. API	FastAPI no Vercel	Valida entrada/conversa, aplica limite por IP e coordena o RAG.
3. Preparação	Gemini ou Groq	Reescreve perguntas de acompanhamento usando histórico, quando necessário.

Etapa	Componente	Responsabilidade
4. Representação	Cloudflare Workers AI BGE-M3	Converte a pergunta em embedding de 1024 dimensões.
5. Recuperação	Cloudflare Vectorize + Oracle	Busca semântica e lexical; combina classificações com RRF.
6. Geração	Gemini; Groq como fallback configurado	Recebe a pergunta e os trechos selecionados e retorna JSON estruturado.
7. Validação	Pydantic + Oracle	Valida resposta e IDs citados, grava interação e devolve links ao widget.

3.1 Caminho de uma pergunta

1. O aluno envia uma pergunta pelo widget do site.
2. A API valida o tamanho, limita a frequência por IP e cria uma conversa nova ou valida o ID e o token existentes.
3. Se há histórico, o modelo pode reescrever a pergunta para deixá-la compreensível sem as mensagens anteriores.
4. BGE-M3 transforma a pergunta em embedding. Em paralelo, o Oracle procura termos no conteúdo.
5. Vectorize devolve trechos semanticamente próximos. O Oracle devolve trechos com palavras correspondentes.
6. A fusão RRF combina as posições das duas buscas. O Oracle fornece o texto integral dos melhores chunks.
7. O modelo responde usando as evidências enviadas. A saída inclui o texto, IDs dos chunks usados e indicação de evidência suficiente.
8. A API descarta IDs que não estavam entre os trechos recuperados, converte os IDs válidos em fontes e grava a interação.
9. O widget mostra resposta e chips de fonte clicáveis. Repetições da mesma página são agrupadas.

Como explicar o RRF

Reciprocal Rank Fusion combina listas pela posição em que cada trecho apareceu, em vez de comparar diretamente scores de naturezas diferentes. A posição combinada serve para ordenar resultados; ela não é, por si só, uma probabilidade de a resposta estar correta.

4. Como o material é preparado

A preparação acontece antes da pergunta. O script de ingestão lê Markdown e notebooks, sem executar células. Ele respeita seções e blocos, forma chunks com sobreposição para preservar contexto e cria URLs com âncoras e IDs determinísticos. Esses IDs permitem reconhecer o mesmo trecho quando o índice é atualizado.

Artefato	Finalidade
scripts/ingest.py	Extrai texto e metadados dos arquivos de aula e produz JSONL.
scripts/embed_with_workers_ai.py	Gera embeddings BGE-M3 em lotes; retoma saída parcial com --resume.
scripts/build_vectorize_ndjson.py	Converte os dados embedados para o formato de upsert do Vectorize.
scripts/load_chunks_oracle.py	Sincroniza chunks, versão e status no Oracle com MERGE para permitir reexecução.
scripts/prune_vectorize.py	Remove do índice vetorial IDs que não existem mais no material atual.
.github/workflows/reindex-rag.yml	Executa sincronização de conteúdo, embeddings, Oracle e Vectorize após mudanças no material.

Na última carga executada, o Wrangler informou que enfileirou 1.465 vetores no índice disruptive-architectures-index. Esse número descreve aquela execução; pode mudar quando o material for atualizado.

5. Banco de dados e proteção da conversa

5.1 Tabelas

Tabela Oracle	Uso
CHUNKS	Texto, título, seção, URL e versão do material.
CONVERSAS	ID da conversa, hash do token e datas.
MENSAGENS	Perguntas e respostas persistidas na ordem da conversa.
INTERACTION_LOGS	Pergunta, resposta, trechos recuperados, latência, tokens e erros.
REINDEX_RUNS	Versões do conteúdo e estado de cada sincronização.

5.2 Token opaco

Ao abrir uma conversa, o backend gera um token aleatório. O navegador recebe o token para poder continuar a conversa, mas o Oracle guarda o hash SHA-256 do token. Em pedidos posteriores, o backend compara o hash recebido ao hash armazenado. Saber apenas o ID da conversa não basta para ler ou continuar aquela sessão.

O widget guarda histórico, ID e token em sessionStorage. Isso preserva a conversa ao navegar entre páginas na mesma aba; fechar a sessão do navegador pode removê-la. A opção Nova conversa apaga o ID e o token da sessão atual e inicia outra conversa.

6. Respostas fundamentadas e troca de modelo

O prompt do sistema define o papel de assistente de estudos e pede saída JSON com resposta, IDs de trechos usados e um booleano que indica se a informação foi encontrada. Pergunta, histórico e documentos são tratados como dados, não como comandos capazes de substituir as regras do sistema.

Depois da geração, Pydantic valida o formato e a aplicação só aceita citações que correspondam a chunks realmente recuperados. Isso reduz fontes inventadas. RAG ajuda a fundamentar respostas, mas não garante verdade absoluta: depende da qualidade dos documentos, da busca e das instruções.

Provedor	Comportamento configurado
Gemini	Principal: gemini-3.8-flash. Em cota, limite ou indisponibilidade temporária, tenta modelos Gemini alternativos configurados.
Groq	Pode ser usado diretamente sem GEMINI_API_KEY ou como fallback para falhas temporárias/quota, se GROQ_API_KEY estiver configurada.

Uma chave inválida não é a mesma coisa que limite de uso: erro 401 de autenticação não é tratado como falha transitória para troca automática de provedor. As chaves ficam em variáveis de ambiente do Vercel ou no ambiente local, nunca no JavaScript público nem no Git.

7. Interface do site e experiência do chat

A identidade visual usa a paleta Atlas de sistemas nos modos claro e escuro, Source Serif 4 para leitura e IBM Plex Mono para código. Navegação, controles e chat usam sans-serif para continuar legíveis. A home apresenta trilhas de IoT e IA em cards; o texto dos labs tem largura de leitura limitada.

- O topo dos labs de GenAI resume objetivos e entregável quando esses dados aparecem no próprio laboratório.
- Os botões das trilhas mantêm texto e borda legíveis no hover e têm foco visível para teclado.
- O widget tem sugestões de perguntas da página, indicador de digitação, foco inicial, fechamento por Esc e suporte a teclado.
- O painel abre como janela flutuante no desktop e bottom sheet no mobile, com safe-area e largura responsiva.
- O botão e o painel podem ser arrastados nas áreas permitidas, encaixam dentro da tela e lembram posição; o menu restaura a posição padrão.
- O cabeçalho é sólido; resposta do assistente fica sem balão e perguntas do aluno aparecem alinhadas à direita.
- Fontes aparecem como chips clicáveis sem repetir a mesma página; mensagens usam scrollbar discreta.
- sessionStorage guarda conversa da aba; localStorage guarda posição do widget.

8. Publicação e demonstração

Parte	Hospedagem	Como conferir
Site e materiais	GitHub Pages	https://kelsonzh0.github.io/DisruptiveArchitectures/
API do chat	Vercel	https://disruptive-architectures-rag.vercel.app/health

O workflow do GitHub Pages publica somente o site estático. O backend é uma função Python no Vercel. `material/js/chat-config.js` escolhe automaticamente a API local na porta 8000 ou a URL Vercel no site publicado. Variáveis de ambiente precisam estar configuradas no backend e alterações nas variáveis exigem novo deploy.

8.1 Demonstração ao professor

1. Abra o site publicado e entre no Lab 4 - RAG e Bases de Conhecimento.
2. Pergunte algo respondido pelo material, como O que é RAG? ou O que são embeddings?
3. Mostre a resposta e abra um chip de fonte para apontar a seção usada.
4. Faça uma segunda pergunta na mesma conversa para demonstrar que o token e o histórico continuam válidos.
5. Pergunte algo fora do escopo, como Qual é a receita de bolo?, e mostre que o assistente informa que não encontrou isso no material.
6. Use Nova conversa para iniciar outra sessão. Não mostre credenciais durante a apresentação.

8.2 Endpoints

Rota	Para que serve	Resultado
GET /health	Verifica se a API está ativa.	JSON com status ok.
GET /docs	Swagger interativo.	Documentação da API.
GET /openapi.json	Esquema OpenAPI.	Documento JSON.
POST /ask	Envia pergunta, histórico e conversa opcionais.	Resposta, fontes e dados da conversa.
GET /conversas/{id}	Consulta mensagens com token no cabeçalho.	Mensagens autorizadas.

Observação

A raiz GET / não foi definida. Um 404 em `http://127.0.0.1:8000/` é esperado; use `/health` ou `/docs`. `/docs%2A%2A` é outro caminho: os asteriscos codificados fazem parte da URL e retornam 404.

9. Problemas encontrados e o que aprendemos

Sintoma	Causa	Correção/aprendizado
/ ou /docs** retorna 404	Raiz sem rota ou URL com asteriscos codificados.	Usar /health, /docs e /openapi.json com caminho exato.
Erro de configuração do banco	Variáveis Oracle ausentes ou backend apontando para banco que não seria usado.	Usar Oracle FIAP e configurar usuário, senha e DSN no backend.
ORA-01017 / ORA-28000	Credencial Oracle inválida e conta bloqueada após tentativas.	Confirmar os dados com a faculdade e solicitar desbloqueio antes de tentar de novo.
ORA-01400 em SECAO	Oracle considera string vazia como NULL; coluna é NOT NULL.	Armazenar um espaço quando o chunk não tem seção.
ORA-00001 em chunks	ID já existia na nova carga.	Usar MERGE e IDs estáveis para atualizar ou inserir sem duplicar.
Cloudflare 401/404	Token sem permissão ou índice Vectorize não criado.	Configurar Workers AI/Vectorize, criar o índice e enviar vetores.
Gemini 400 com schema	Formato estruturado incompatível com o campo aceito.	Usar o schema suportado pelo SDK e validar a resposta.
Erro de tamanho de embedding	Lote ou texto ultrapassou o limite do modelo.	Usar lotes menores, localizar o chunk problemático e retomar com --resume.
Segunda pergunta recebia HTTP 401	Inserção da conversa sem commit era desfeita ao fechar a conexão; o ID de OUT bind também foi normalizado.	Confirmar transações ao criar conversas e salvar interações. Commit publicado: 741de3d.
MkDocs não instalado	Ambiente Python sem dependências do site.	Instalar docs-requirements.txt no ambiente que serve o MkDocs.

10. Arquivos que vale saber explicar

Caminho	Papel
backend/app/main.py	Monta FastAPI, CORS, Oracle, Vectorize, embeddings, modelo e limite de requisições.
backend/app/api/routes.py	Implementa health, ask, proteção da conversa e erros HTTP.
backend/app/services/rag.py	Orquestra reescrita, embedding, busca, fusão, LLM e validação de citações.
backend/app/services/llm.py	Prompt, schema estruturado, Gemini, fallbacks e Groq.
backend/app/repositories/oracle.py	Persistência Oracle, token hash, busca lexical e histórico.
backend/app/repositories/vectorize.py	Consulta o índice vetorial Cloudflare.
backend/app/services/embeddings.py	Gera embedding da pergunta com Workers AI BGE-M3.
backend/app/config.py	Lê e valida variáveis de ambiente sem expor segredos.
backend/app/schemas/chat.py	Define contratos de entrada e saída com Pydantic.
backend/migrations/oracle_001_initial_schema.sql	Cria tabelas e índices do Oracle.
material/js/chat-widget.js	Interface, sessão, estado, acessibilidade, fontes e interações.
material/js/chat-config.js	Escolhe URL local ou API Vercel.
material/stylesheets/extra.css	Paleta, tipografia, layout responsivo e estilos.
mkdocs.yml	Tema Material, navegação, extensões e arquivos do chat.
vercel.json e index.py	Configuração e entrada da API no Vercel.

11. Roteiro oral de dois minutos

Fala sugerida

Nosso projeto é um chat RAG integrado ao material do curso. A interface está no GitHub Pages e a API Python/FastAPI está no Vercel. Primeiro, o material Markdown e os notebooks são divididos em chunks sem executar as células. Cada chunk recebe metadados e um embedding BGE-M3 de 1024 dimensões. O texto fica no Oracle e os vetores no Cloudflare Vectorize.

Quando alguém pergunta, o backend transforma a pergunta em embedding e faz duas buscas: uma semântica no Vectorize e outra lexical no Oracle. O RRF combina as posições. O modelo Gemini recebe os melhores trechos e devolve JSON com a resposta e os IDs das evidências. O backend valida esses IDs e só então apresenta as fontes no chat.

O ponto central é que o modelo responde com o contexto recuperado do material e deve avisar quando não encontra evidência suficiente. O projeto está publicado e testamos perguntas em sequência. Para manter a conversa no Oracle, também confirmamos explicitamente as transações.

12. Perguntas que o professor pode fazer

Pergunta	Resposta curta
Por que usar RAG em vez de só perguntar ao LLM?	Recupera contexto específico do curso, reduz respostas fora do material e aponta fontes.
Por que combinar busca vetorial e lexical?	A vetorial encontra significado próximo; a lexical encontra termos explícitos.
O que faz o RRF?	Combina rankings sem comparar diretamente escalas de score incompatíveis.
RAG elimina alucinações?	Não. Ajuda a fundamentar, mas depende de dados, busca e instruções.
Por que Oracle e Vectorize?	Oracle guarda texto e sessões; Vectorize compara vetores semanticamente.
Por que a segunda pergunta dava 401?	A conversa era revertida ao fechar a conexão por falta de commit; o backend passou a confirmar a transação.
E se não houver resposta no material?	O assistente informa a ausência de evidência e não deve inventar fonte.
Como as chaves são protegidas?	Ficam em variáveis de ambiente no backend, não no JavaScript nem no repositório.

13. Limitações e próximos passos

- A busca lexical é uma alternativa simples baseada em termos; não usa Oracle Text.
- O limitador atual guarda estado em memória por instância. Escalar para várias instâncias pede um armazenamento compartilhado.
- A qualidade depende dos documentos, do tamanho dos chunks, do embedding, da recuperação e do modelo.
- Gemini, Groq, Cloudflare e Oracle têm cotas e credenciais próprias.
- Uma próxima etapa útil é avaliar perguntas conhecidas com fontes esperadas e métricas de recuperação e resposta.
- O texto de erro pode distinguir token de conversa inválido de falha de autenticação do provedor.

14. Referências do projeto

- Site: <https://kelsonzh0.github.io/DisruptiveArchitectures/>
- API: <https://disruptive-architectures-rag.vercel.app/health>
- Repositório: <https://github.com/KelsonZh0/DisruptiveArchitectures>
- Commits relevantes: 38d5217 conectou chat e Vercel; 2d51e7e publicou o fork; 44228f7 deduplicou fontes por página; f895607 normalizou o ID do Oracle; 741de3d confirmou gravação da conversa e interações.
- Esta apostila descreve o estado implementado. arquitetura-rag.md registra a proposta inicial e pode mencionar decisões anteriores, como uma hospedagem que depois mudou para Vercel.